



Jerry Lin <jerry@museder.com>

[WordPress Plugin Directory] Review in Progress: Museder RestoreOne

WordPress.org Plugin Directory <plugins@wordpress.org>

2026年1月24日 凌晨3:11

收件者: Jerry Lin <jerry@museder.com>

🙌 Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please check this email thoroughly, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

AI generated output.

We've detected files that appear to be the output of an AI tool used during development.

These files usually include explanations or summaries of the changes made by the AI. That's great! We appreciate the effort you're putting into improving your plugin. However, those files aren't necessary for the plugin functionality.

They are also typically not useful as documentation because they tend to be partial and unconnected files that are also not maintained or updated.

We encourage you to use the available tools to improve your plugin. However, please remove any unnecessary AI-generated output. This will help keep the plugin package clean and reduce potential confusion for users and reviewers.

From your plugin:

museder-restoreone/docs/wp-compliance-checklist.md

Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', \$options);
- add_shortcode('musere_shortcode', \$callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global \$musere_options;
- add_action('wp_ajax_musere_save_data', ...);

- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (!function_exists('NAME'))` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonereports",
"musederrestoreonerestore", "musederrestoreoneadminui" for 22 element(s).
# This plugin is using the prefix "backup_lite" for 146 element(s).
```

```
# Looks like there are elements not using common prefixes.
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Allowing direct file access to plugin files

Direct file access occurs when someone directly queries a PHP file. This can be done by entering the complete path to the file in the browser's URL bar or by sending a POST request directly to the file.

For files that only contain class or function definitions, the risk of something funky happening when accessed directly is minimal. However, for files that contain executable code (e.g., function calls, class instance creation, class method calls, or inclusion of other PHP files), the risk of security issues is hard to predict because it depends on the specific case, but it can exist and it can be high.

You can easily prevent this by adding the following code at the beginning of all PHP files that could potentially execute code if accessed directly:

```
if ( ! defined( 'ABSPATH' ) ) exit; // Exit if accessed directly
```

Add it after the `<?php` opening tag and after the namespace declaration, if any, but before any other code.

Example(s) from your plugin:

download-handler.php:19

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

- ✓ I fixed all the issues in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. I am familiar with tools such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.
- ✓ I tested my updated plugin on a clean WordPress installation with [WP_DEBUG set to true](#).

⚠️ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

- ✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.
- ✓ I went to ["Add your plugin"](#) and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*
- ✓ I replied to this email. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

📢 To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, we ask you to thoroughly check all shared issues and fix them before sending the code back to us. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T7 23Jan26/3.8RC2

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>